

Cats: Typklassen & Funktionale Abstraktion

Cats: Typklassen & Funktionale Abstraktion

Gliederung



- 1. Grundlagen (Semigroup, Monoid, HKT)
- 2. Cats einrichten & Show, Eq, Order
- 3. Functor, Applicative, Monad
- 4. Traverse
- 5. Validated & NonEmptyList
- 6. Monad Transformers & Kleisli
- 7. Übung

Theoretische Grundlagen

Theoretische Grundlagen

Woher kommt Cats?



- Cats steht für **Categories**, inspiriert von der Kategorientheorie
- Kategorientheorie: Mathematik die Strukturen und deren Beziehungen beschreibt
 - Objekte (Typen), Morphismen (Funktionen), Komposition, Identität
- Für uns relevant: daraus entstehen **Typklassen mit Gesetzen**
 - Functor, Monad, Semigroup, Monoid, ...
- Cats codiert diese Muster als Scala-Typklassen mit Standardinstanzen

Theoretische Grundlagen

Semigroup & Monoid



- **Semigroup:** eine Operation ``combine`` (``|+|``), Gesetz: Assoziativität
- Wichtig für Cats: Validated akkumuliert Fehler mit Semigroup

```
import cats.Monoid
import cats.syntax.semigroup._

1 |+| 2 |+| 3           // 6
"Hello" |+| " World"    // "Hello World"
List(1,2) |+| List(3,4) // List(1,2,3,4)
```

Theoretische Grundlagen

Semigroup für eigene Typen

- Nützlich für: Logs zusammenfassen, Metriken akkumulieren, Configs mergen
- Semigroup kann **keine leere/neutrale** Instanz liefern

```
case class Score(points: Int, bonus: Int)
```

```
given Semigroup[Score] = Semigroup.instance { (a, b) =>  
  Score(a.points + b.points, a.bonus + b.bonus)  
}
```

```
Score(10, 2) |+| Score(5, 1) // Score(15, 3)
```

Theoretische Grundlagen

Monoid: Semigroup + empty

- **Monoid** = Semigroup + `empty` (neutrales Element)
- Ermöglicht sichere Aggregation auch bei leeren Collections:
- Ohne Monoid: `list.reduce(\_ |+| \_)` crasht bei leerer Liste

```
import cats.Monoid
```

```
Monoid[Int].empty      // 0
```

```
Monoid[String].empty   // ""
```

```
List(1, 2, 3, 4).combineAll // 10
```

```
List.empty[Int].combineAll // 0 (kein Crash!)
```

- Maps mit gleichem Key automatisch zusammenführen:
- Cats liefert `Monoid[Map[K, V]]` wenn `V` ein Monoid hat
- Perfekt für: Metriken aggregieren, Config-Overrides, Feature-Flags

```
val day1 = Map("errors" -> 3, "requests" -> 100)
val day2 = Map("errors" -> 1, "requests" -> 150, "timeouts" -> 2)

day1 |+| day2
// Map("errors" -> 4, "requests" -> 250, "timeouts" -> 2)
```


Theoretische Grundlagen

Warum eine Library?



- In VL04 haben wir eigene Typklassen gebaut (Show, JsonWriter)
- Problem: jedes Projekt erfindet das Rad neu
- Cats liefert:
 - Standardisierte Typklassen mit geprüften Gesetzen
 - Instanzen für Standardtypen (Option, List, Either, ...)
 - Syntax-Extensions (``.show``, ``.map``, ``.flatMap``, ...)
 - Interoperabilität zwischen Libraries (http4s, doobie, fs2, ...)

Theoretische Grundlagen

Cats einbinden

- build.sbt:
- Typische Imports:

```
libraryDependencies += "org.typelevel" %% "cats-core" % "2.12.0"
```

```
import cats._  
import cats.implicits._  
// oder gezielt:  
import cats.syntax.show._
```

Higher-Kinded Types

Higher-Kinded Types

Das Problem

- Wir wollen eine Typklasse für "Dinge die man mappen kann"
- List, Option, Future: alle haben `map`
- Aber wie beschreiben wir den **Container-Typ** generisch?

```
// Das geht NICHT:  
trait Mappable[Container] {  
  def map[A, B](fa: Container[A])(f: A => B): Container[B]  
}  
// Fehler: Container erwartet keine Typparameter
```

Higher-Kinded Types

Kinds: Typen von Typen

- **Kind** beschreibt die "Signatur" eines Typs:
 - ``Int`, `String`` $\rightarrow$ Kind ``*`` (konkreter Typ)
 - ``List`, `Option`` $\rightarrow$ Kind ``*`` $\rightarrow$ ``*`` (Typkonstruktor, braucht 1 Argument)
 - ``Map`, `Either`` $\rightarrow$ Kind ``*`` $\rightarrow$ ``*`` $\rightarrow$ ``*`` (braucht 2 Argumente)
- Scala-Syntax für höhere Kinds: **`F[_]`**

```
trait Functor[F[_]] {  
  def map[A, B](fa: F[A])(f: A => B): F[B]  
}
```

Higher-Kinded Types

Was bringt uns $F[_]$?

- **Eine** Funktion die mit **jedem** Container funktioniert:
- Ohne $F[_]$ bräuchten wir drei separate Funktionen
- HKTs ermöglichen **generische Programmierung über Kontexte**

```
def increment[F[_]: Functor](fa: F[Int]): F[Int] =  
  fa.map(_ + 1)
```

```
increment(Option(41))    // Some(42)  
increment(List(1,2,3))   // List(2,3,4)  
increment(Future(99))    // Future(100)
```

Higher-Kinded Types

Welche Typklassen nutzen $F[_]$?



- Die wichtigsten HKT-Typklassen in Cats:
 - **Functor**[F] $\rightarrow$ ``map``: Wert im Container transformieren
 - **Applicative**[F] $\rightarrow$ ``pure`` + ``mapN``: Werte unabhängig kombinieren
 - **Monad**[F] $\rightarrow$ ``flatMap``: sequenzielle Verkettung
 - **Traverse**[F] $\rightarrow$ ``traverse``/``sequence``: Effekte einsammeln
- Jede erweitert die vorherige: Functor < Applicative < Monad

Higher-Kinded Types

Either und type lambda

- Either hat Kind $`* \rightarrow * \rightarrow *`$, passt also nicht auf $F[_]$
- Lösung: einen Parameter fixieren

```
// Type Lambda Syntax (Scala 3):
```

```
type EitherString = [A] =>> Either[String, A]
```

```
val eitherFunctor: Functor[EitherString] = new Functor[EitherString] {  
  def map[A, B](fa: Either[String, A])(f: A => B): Either[String, B] =  
    fa.map(f)  
}
```

```
// Kurzform mit type alias:
```

```
type StringOr[A] = Either[String, A]
```

```
// Jetzt: Functor[StringOr]
```


Cats: Show, Eq, Order

- Problem mit `toString`: jedes Objekt hat es, oft nutzlos
- Cats' `Show[A]` erzwingt eine **explizite** Darstellung
- Vorteil: Compiler-Fehler wenn kein Show definiert ist
 - Kein versehentliches `ClassName@3f2a1b` in Logs

```
import cats.Show
import cats.syntax.show._
```

```
given Show[Int] = Show.show(n => s"Zahl($n)")
```

```
42.show // "Zahl(42)", nicht einfach "42"
```

Cats: Show, Eq, Order

Show für eigene Typen

- Eigene Instanz definieren:
- Oder via `cats.derived` automatisch ableiten (mit Kittens: `derives Show`)

```
case class Student(name: String, semester: Int)
```

```
given Show[Student] = Show.show { s =>  
  s"${s.name} (Semester ${s.semester})"  
}
```

```
val anna = Student("Anna", 3)  
anna.show // "Anna (Semester 3)"
```

Cats: Show, Eq, Order

Eq: typsichere Gleichheit

- Problem: `==` vergleicht alles mit allem, auch Unsinn
- Cats' `Eq[A]` erzwingt gleiche Typen:

```
"hello" == 42 // kompiliert! Ergebnis: false
```

```
import cats.Eq
import cats.syntax.eq._
```

```
given Eq[Int] = Eq.fromUniversalEquals
```

```
42 === 42 // true
```

```
42 != 99 // true
```

```
// "hello" === 42 // Kompilierfehler!
```

Cats: Show, Eq, Order

Eq für eigene Typen

- Für case classes oft: `Eq.fromUniversalEquals`

```
case class Student(name: String, semester: Int)
```

```
given Eq[Student] = Eq.instance { (a, b) =>  
  a.name == b.name && a.semester == b.semester  
}
```

```
val s1 = Student("Anna", 3)
```

```
val s2 = Student("Anna", 3)
```

```
val s3 = Student("Bob", 5)
```

```
s1 === s2 // true
```

```
s1 === s3 // false
```

Cats: Show, Eq, Order

Order: typsichere Sortierung

- Erweitert Eq um Vergleichsoperationen:

```
import cats.Order
import cats.syntax.order._
```

```
given Order[Int] = Order.fromOrdering[Int]
```

```
42.compare(99) // negative Zahl (kleiner)
```

```
42.max(99)     // 99
```

```
42.min(99)     // 42
```

```
val students: List[Student] = ???
```

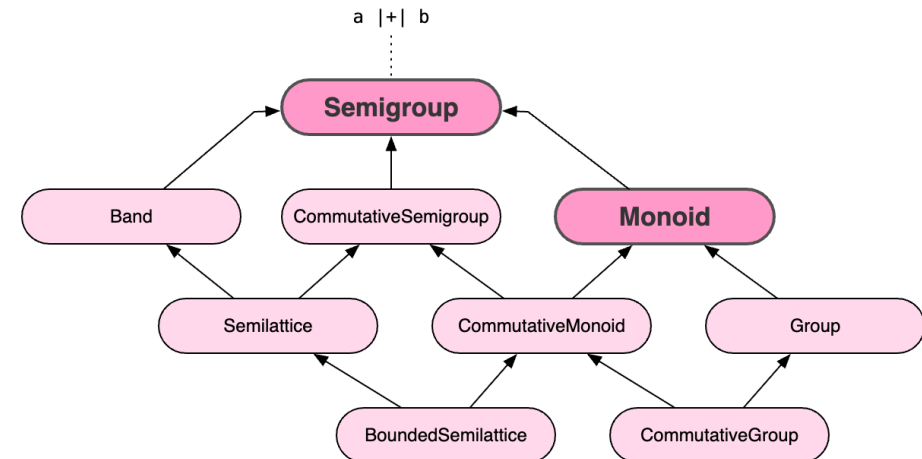
```
students.sorted(Order[Student].toOrdering)
```

Cats: Show, Eq, Order

Typklassen-Hierarchie in Cats

- Cats organisiert Typklassen in einer **Vererbungshierarchie**:

- Eq $\rightarrow$ PartialOrder $\rightarrow$ Order
- Functor $\rightarrow$ Applicative $\rightarrow$ Monad (nächste Seite)
- Semigroup $\rightarrow$ Monoid

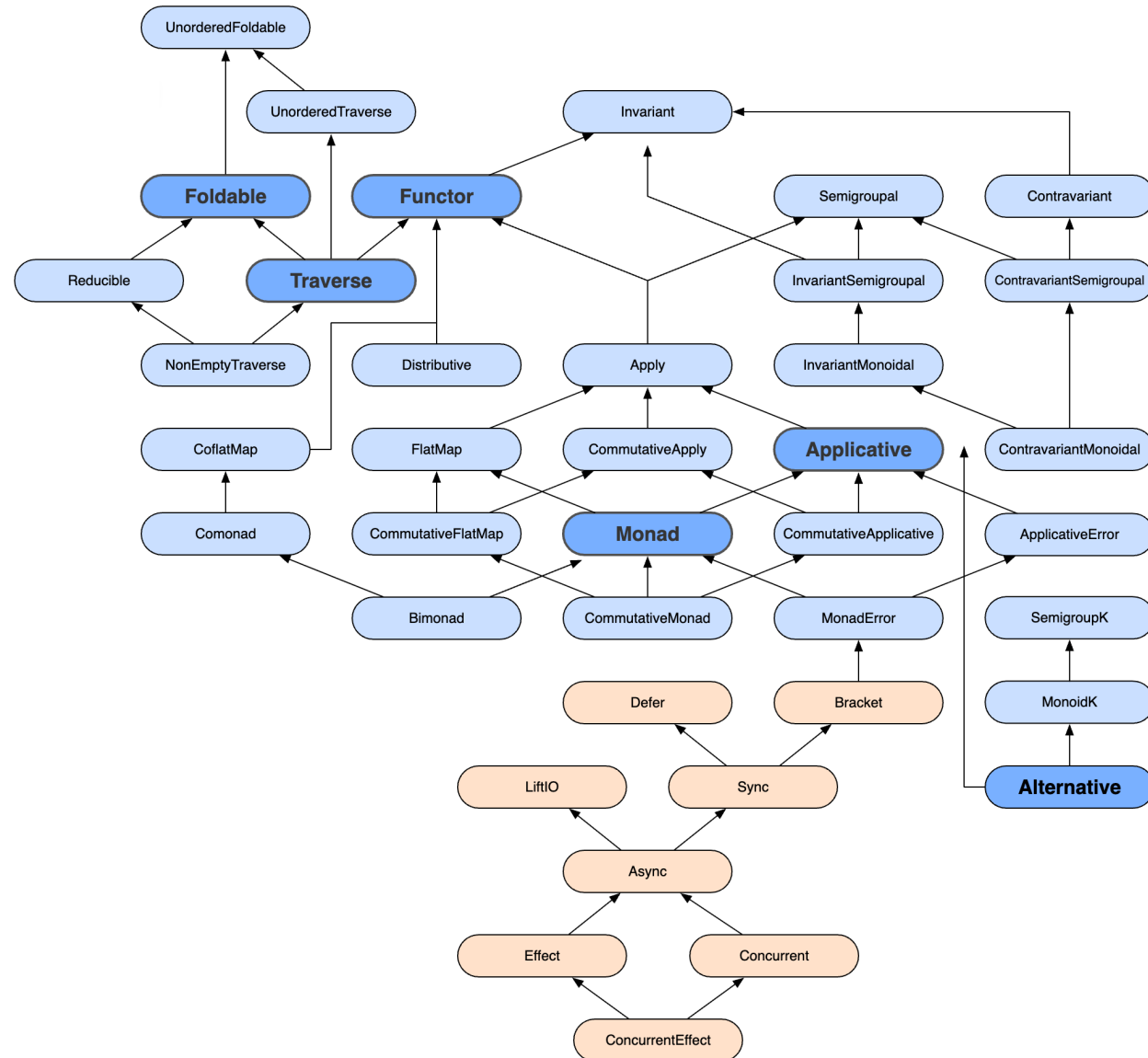


- Warum wichtig?
 - Wer `Monad[F]` hat, bekommt `Functor[F]` gratis
 - Code der nur `Functor` braucht, akzeptiert auch `Monad`
 - Prinzip: **minimale Anforderung** in Signaturen verwenden

Cats: Show, Eq, Order

Typklassen-Hierarchie in Cats

- NICHT AUSWENDIG LERNEN!
- NUR ZUR VERANSCHAULICHUNG!



Cats: Show, Eq, Order

Cheat-Sheet: Was bringt was?



- **Semigroup** → `|+|` kombiniert zwei Werte
- **Monoid** → `|+|` + `empty` + `combineAll`
- **Functor** → `map`: Wert im Container transformieren
- **Applicative** → `pure` + `mapN`: mehrere unabhängige Werte kombinieren
- **Monad** → `flatMap` + `for`: sequenzielle Verkettung
- **Traverse** → `traverse` / `sequence`: Collection von Effekten umdrehen
- **Eq/Order** → `===`, `!=`, `compare`, `max`, `min`
- **Show** → `.show`: typsichere String-Ausgabe

Functor, Applicative, Monad

Functor, Applicative, Monad

Functor in Cats



- In der HKT-Section haben wir Functor schon gesehen
- Cats liefert Functor-Instanzen für alle Standardtypen:
- **Functor-Gesetze:**
 - ``fa.map(identity) == fa`` (Identität)
 - ``fa.map(f).map(g) == fa.map(f andThen g)`` (Komposition)

```
import cats.Functor
import cats.syntax.functor._
```

```
Option(5).map(_ * 2)           // Some(10)
List(1, 2, 3).map(_ + 10)      // List(11, 12, 13)
Right(42).map(_ * 2)           // Right(84)
```

Functor, Applicative, Monad

Applicative: unabhängige Effekte



- Problem: ``map`` transformiert nur **einen** Wert im Container
- Was wenn wir **mehrere unabhängige** Werte kombinieren wollen?
- Applicative löst das mit ``pure`` (Wert in Container heben) und ``ap`/`mapN``

```
import cats.Applicative
```

```
// pure: einen Wert in den Kontext heben  
Applicative[Option].pure(42)  // Some(42)  
Applicative[List].pure(42)    // List(42)
```

Functor, Applicative, Monad

mapN: mehrere Werte kombinieren



- `mapN` nimmt ein Tupel von F-Werten und eine Funktion:
- Funktioniert mit 2 bis 22 Werten
- Wenn **einer** None ist → gesamtes Ergebnis ist None

```
import cats.syntax.apply._
```

```
val name: Option[String] = Some("Anna")
```

```
val age: Option[Int] = Some(23)
```

```
val city: Option[String] = Some("Rosenheim")
```

```
(name, age, city).mapN { (n, a, c) =>  
  s"$n, $a Jahre, aus $c"  
}
```

```
// Some("Anna, 23 Jahre, aus Rosenheim")
```

Functor, Applicative, Monad

mapN: Fehlerfall



- Bei Option: erster None-Wert macht alles None
- Bei Either: erster Left-Wert wird zum Ergebnis
- Bei **Validated**: alle Fehler werden gesammelt (dazu später mehr)

```
val name: Option[String] = Some("Anna")
val age: Option[Int] = None
val city: Option[String] = Some("Rosenheim")
```

```
(name, age, city).mapN { (n, a, c) =>
  s"$n, $a Jahre, aus $c"
}
// None
```

Functor, Applicative, Monad

Applicative vs Monad



- **Applicative:** Effekte sind **unabhängig** voneinander
 - Beide Seiten werden ausgewertet, dann kombiniert
 - Ermöglicht parallele Auswertung
- **Monad:** Effekte sind **sequenziell abhängig**
 - Zweiter Wert hängt vom Ergebnis des ersten ab
 - flatMap: ``F[A] => (A => F[B]) => F[B]``

```
// Monad: zweites Future hängt vom ersten ab
for {
  userId <- findUser(name)
  orders <- loadOrders(userId) // braucht userId!
} yield orders
```

Functor, Applicative, Monad

Monad in Cats



- Monad kennen wir aus VL03 (flatMap, for-Comprehension)
- Cats definiert `Monad[F]` mit `pure` und `flatMap`:

```
import cats.Monad
// Generische Funktion über beliebige Monaden:
def compose[F[_]: Monad, A, B, C](
  f: A => F[B], g: B => F[C]
): A => F[C] =
  a => f(a).flatMap(g)

val parse: String => Option[Int] = _.toIntOption
val half: Int => Option[Int] = n => if (n % 2 == 0) Some(n / 2) else None

compose(parse, half)("42") // Some(21)
compose(parse, half)("abc") // None
```


Functor, Applicative, Monad

Monad-Gesetze (Wiederholung)

- **Left Identity:** ``pure(a).flatMap(f) == f(a)``
- **Right Identity:** ``m.flatMap(pure) == m``
- **Assoziativität:** ``m.flatMap(f).flatMap(g) == m.flatMap(a => f(a).flatMap(g))``
- Cats stellt ``cats-laws`` bereit um diese automatisch zu testen:

```
// In Tests mit ScalaCheck:  
import cats.laws.discipline.MonadTests  
  
checkAll("Option.MonadLaws",  
  MonadTests[Option].monad[Int, Int, Int]  
)
```

Functor, Applicative, Monad

Traverse: Effekte einsammeln



- Häufiges Problem: eine Liste von Effekten zu einem Effekt über eine Liste machen
- ``List[Option[A]]` → `Option[List[A]]` (nur Some wenn alle Some)`
- Noch mächtiger: ``.traverse`` kombiniert `map` + `sequence`

```
import cats.syntax.traverse._
```

```
val opts: List[Option[Int]] = List(Some(1), Some(2), Some(3))  
opts.sequence // Some(List(1, 2, 3))
```

```
val withNone: List[Option[Int]] = List(Some(1), None, Some(3))  
withNone.sequence // None
```

- ``traverse(f)`` wendet ``f: A => F[B]`` an und sammelt die Ergebnisse:
- Funktioniert mit jeder ``Applicative``: `Option`, `Either`, `Validated`, `Future`, ...

```
def parseInt(s: String): Option[Int] = s.toIntOption
```

```
val strings = List("1", "2", "3")  
strings.traverse(parseInt) // Some(List(1, 2, 3))
```

```
val bad = List("1", "abc", "3")  
bad.traverse(parseInt) // None
```

Functor, Applicative, Monad

Traverse mit Future: parallele Calls



- Mehrere HTTP-Requests parallel ausführen und Ergebnisse sammeln:
- Ohne traverse: manuell `Future.sequence(userIds.map(fetchUser))`
- `traverse` = `map` + `sequence` in einem Schritt

```
def fetchUser(id: Int): Future[User] = ???
```

```
val userIds = List(1, 2, 3, 4, 5)
```

```
val allUsers: Future[List[User]] = userIds.traverse(fetchUser)
```

```
// Alle Futures laufen parallel, Ergebnis ist Future[List[User]]
```

- Jedes Element validieren und **alle** Fehler auf einmal bekommen:

```
def validatePositive(n: Int): ValidatedNel[String, Int] =  
  if (n > 0) Validated.valid(n)  
  else Validated.invalidNel(s"$n ist nicht positiv")  
  
val numbers = List(1, -2, 3, -4)  
numbers.traverse(validatePositive)  
// Invalid(NonEmptyList("-2 ist nicht positiv", "-4 ist nicht positiv"))  
  
val good = List(1, 2, 3)  
good.traverse(validatePositive)  
// Valid(List(1, 2, 3))
```

Validated & NonEmptyList

Validated & NonEmptyList

Das Problem mit Either

- Either ist ein Monad $\rightarrow$ sequenziell (short-circuit)
- Beim ersten Fehler wird abgebrochen:

```
def validateName(s: String): Either[String, String] =  
  if (s.nonEmpty) Right(s) else Left("Name leer")
```

```
def validateAge(n: Int): Either[String, Int] =  
  if (n > 0) Right(n) else Left("Alter ungültig")
```

Validated & NonEmptyList

Either: Short-Circuit in Aktion

- for bricht beim ERSTEN Fehler ab:
- In Formularen will man **alle** Fehler auf einmal zeigen
- Either kann das strukturell nicht, wegen flatMap/Monad

```
for {  
  name <- validateName("")  
  age  <- validateAge(-1)    // wird nie erreicht!  
} yield (name, age)  
// Left("Name leer"), zweiter Fehler geht verloren
```


Validated & NonEmptyList

Validated: alle Fehler sammeln



- `Validated[E, A]` ist **kein Monad**, und zwar absichtlich!
- Stattdessen: Applicative → unabhängige Validierungen
- **Wichtig:** E braucht eine `Semigroup`-Instanz zur Fehler-Akkumulation

```
import cats.data.Validated
import cats.data.Validated.{Valid, Invalid}
import cats.syntax.apply._

type ValidationResult[A] = Validated[List[String], A]

val name: ValidationResult[String] = Invalid(List("Name leer"))
val age: ValidationResult[Int] = Invalid(List("Alter ungültig"))

(name, age).mapN((n, a) => (n, a))
// Invalid(List("Name leer", "Alter ungültig"))
```

Validated & NonEmptyList

NonEmptyList (NEL)

- Problem: `List[String]` als Fehlertyp erlaubt leere Liste
- `NonEmptyList[A]` garantiert mindestens ein Element:

```
import cats.data.NonEmptyList
import cats.data.ValidatedNel
```

```
type Result[A] = ValidatedNel[String, A] // = Validated[NonEmptyList[String], A]
```

```
def validateName(s: String): Result[String] =
  if (s.nonEmpty) Validated.valid(s)
  else Validated.invalidNel("Name leer")
```

```
def validateAge(n: Int): Result[Int] =
  if (n > 0) Validated.valid(n)
  else Validated.invalidNel("Alter ungültig")
```

Validated & NonEmptyList

Validated in der Praxis



```
import cats.syntax.apply._

case class Registration(name: String, age: Int, email: String)

def validateEmail(s: String): Result[String] =
  if (s.contains("@")) Validated.valid(s)
  else Validated.invalidNel("Keine gültige E-Mail")

val result = (
  validateName(""),
  validateAge(-1),
  validateEmail("invalid")
).mapN(Registration.apply)

// Invalid(NonEmptyList("Name leer", "Alter ungültig", "Keine gültige E-Mail"))
```

Validated & NonEmptyList

Ior: Fehler UND Erfolg gleichzeitig

- `Either`: nur Links ODER Rechts
- `Validated`: sammelt Fehler, aber bei Erfolg keine Warnungen
- `Ior[A, B]` hat **drei** Fälle:
 - `Ior.Left(a)` → nur Fehler
 - `Ior.Right(b)` → nur Erfolg
 - `Ior.Both(a, b)` → Erfolg MIT Warnungen/Fehlern

```
import cats.data.Ior
```

```
val success: Ior[String, Int] = Ior.right(42)
```

```
val warning: Ior[String, Int] = Ior.both("Achtung: Wert gerundet", 42)
```

```
val failure: Ior[String, Int] = Ior.left("Fataler Fehler")
```

Validated & NonEmptyList Ior in der Praxis

- Mit `IorNel` (= `Ior[NonEmptyList[String], A]`) Warnungen sammeln:

```
import cats.data.IorNel
import cats.syntax.all._

def parseInt(s: String): IorNel[String, Int] =
  s.toIntOption match {
    case Some(n) if n < 0 => Ior.both(NonEmptyList.one("Negativer Wert"), n)
    case Some(n)          => Ior.right(n)
    case None              => Ior.left(NonEmptyList.one(s"'$s' ist keine Zahl"))
  }

parseInt("42")    // Right(42)
parseInt("-5")    // Both(NEL("Negativer Wert"), -5)
parseInt("abc")   // Left(NEL("'abc' ist keine Zahl"))
```

Validated & NonEmptyList

Validated ↔ Either wechseln

- Manchmal braucht man beides:
 - Validated für unabhängige Checks
 - Either für sequenzielle Abhängigkeiten

```
val validated: ValidatedNel[String, Int] =  
    Validated.valid(42)  
  
// Validated → Either  
val either: Either[NonEmptyList[String], Int] =  
    validated.toEither  
  
// Either → Validated  
val backToValidated =  
    either.toValidated
```

Monad Transformers

Monad Transformers

Das Verschachtelungsproblem

- Realer Code hat oft `Future[Either[Error, A]]`:
- 3 Schritte, jeder kann fehlschlagen (Left) oder erfolgreich sein (Right)
- Wie verkettet man das?

```
def findUser(id: Int): Future[Either[String, User]] = ???  
def loadProfile(user: User): Future[Either[String, Profile]] = ???  
def loadSettings(profile: Profile): Future[Either[String, Settings]] = ???
```


Monad Transformers

Stufe 1: ein Schritt



```
val result: Future[Either[String, User]] = findUser(1)
// Noch einfach, kein Problem
```

Monad Transformers

Stufe 2: zwei Schritte

- Schon hässlich: manuelles Pattern-Match + Fehler durchreichen

```
val result: Future[Either[String, Profile]] =  
  findUser(1).flatMap {  
    case Right(user) => loadProfile(user)  
    case Left(err)   => Future.successful(Left(err))  
  }
```

Monad Transformers

Stufe 3: drei Schritte

- Verschachtelung wächst mit jedem Schritt
- `Future.successful(Left(err))` wiederholt sich überall
- Bei 5+ Schritten: unlesbar, fehleranfällig

```
val result: Future[Either[String, Settings]] =
  findUser(1).flatMap {
    case Right(user) => loadProfile(user).flatMap {
      case Right(profile) => loadSettings(profile)
      case Left(err)      => Future.successful(Left(err))
    }
    case Left(err) => Future.successful(Left(err))
  }
```

Monad Transformers

Die Lösung: EitherT



- `EitherT[F, E, A]` wickelt `F[Either[E, A]]` und abstrahiert das Pattern-Match weg:

```
import cats.data.EitherT

def findUser(id: Int): EitherT[Future, String, User] = EitherT(db.findUser(id))
def loadProfile(user: User): EitherT[Future, String, Profile] = EitherT(profileService.load(user))
def loadSettings(profile: Profile): EitherT[Future, String, Settings] =
    EitherT(settingsService.load(profile))

val result: EitherT[Future, String, Settings] = for {
  user      <- findUser(1)
  profile   <- loadProfile(user)
  settings  <- loadSettings(profile)
} yield settings
```

Monad Transformers

EitherT: realistisches Szenario

- Typische Service-Pipeline: DB → Validierung → externer Call:

```
type AppError = String
type Result[A] = EitherT[Future, AppError, A]

def findOrder(id: Int): Result[Order] =
  EitherT.fromOptionF(db.findById(id), "Order nicht gefunden")

def validateStock(order: Order): Result[Order] =
  EitherT.cond(order.quantity > 0, order, "Menge ungültig")

def chargePayment(order: Order): Result[Receipt] =
  EitherT(paymentService.charge(order))
```

- Bei **jedem** Schritt: wenn Left, bricht die Pipeline ab
- Kein verschachteltes Pattern-Matching nötig

```
def processOrder(id: Int): Result[Receipt] = for {  
  order    <- findOrder(id)  
  valid    <- validateStock(order)  
  receipt <- chargePayment(valid)  
} yield receipt
```

```
// Aufruf:  
processOrder(42).value.map {  
  case Right(receipt) => s"Bezahlt: ${receipt.amount}"  
  case Left(error)    => s"Fehler: $error"  
}
```

Monad Transformers

Transformer-Helfer



- Werte in den Transformer heben:

```
// Reiner Wert → OptionT
```

```
val pure: OptionT[Future, Int] = OptionT.pure(42)
```

```
// F[A] → OptionT (immer Some)
```

```
val lifted: OptionT[Future, Int] =  
  OptionT.liftF(Future.successful(42))
```

```
// Option[A] → OptionT
```

```
val fromOption: OptionT[Future, Int] =  
  OptionT.fromOption[Future](Some(42))
```

```
// None → OptionT
```

```
val none: OptionT[Future, Int] = OptionT.none[Future, Int]
```

- Normale Komposition: ``A => B`` und ``B => C`` ergibt ``A => C``
- Problem: ``A => F[B]`` und ``B => F[C]``, wie komponieren?
- **Kleisli[F, A, B]** wickelt ``A => F[B]`` und erlaubt Komposition:

```
import cats.data.Kleisli
```

```
val parse: Kleisli[Option, String, Int] = Kleisli(s => s.toIntOption)
```

```
val isEven: Kleisli[Option, Int, Boolean] = Kleisli(n => Some(n % 2 == 0))
```

```
val parseAndCheck = parse.andThen(isEven)
```

```
parseAndCheck.run("42")    // Some(true)
```

```
parseAndCheck.run("abc")    // None
```


Monad Transformers

Kleisli: Dependency Injection



- Kleisli eignet sich für **Reader-Pattern** (Konfiguration durchreichen):
- ``Reader[A, B]`` ist eigentlich ``Kleisli[Id, A, B]`` (wobei ``type Id[A] = A``)

```
case class Config(dbUrl: String, apiKey: String)

val getDbUrl: Kleisli[Option, Config, String] = Kleisli(cfg => Some(cfg.dbUrl))

val connect: Kleisli[Option, String, String] = Kleisli(url => Some(s"Connected to $url"))

val pipeline: Kleisli[Option, Config, String] = getDbUrl.andThen(connect)

pipeline.run(Config("jdbc:h2:mem", "secret"))
// Some("Connected to jdbc:h2:mem")
```

Monad Transformers

Kleisli: Validierungs-Pipeline



- Mehrere Validierungsschritte als Kleisli komponieren:

```
val trimInput: Kleisli[Either[String, *], String, String] =  
  Kleisli(s => Right(s.trim))
```

```
val checkNonEmpty: Kleisli[Either[String, *], String, String] =  
  Kleisli(s => if (s.nonEmpty) Right(s) else Left("Darf nicht leer sein"))
```

```
val checkLength: Kleisli[Either[String, *], String, String] =  
  Kleisli(s => if (s.length >= 3) Right(s) else Left("Mind. 3 Zeichen"))
```

```
val validateUsername = trimInput.andThen(checkNonEmpty).andThen(checkLength)  
validateUsername.run(" ab ") // Left("Mind. 3 Zeichen")  
validateUsername.run(" Anna ") // Right("Anna")
```

Monad Transformers

Wann welchen Ansatz?



- **Kein Transformer nötig:** nur eine Monade (nur Future, nur Option)
- **OptionT:** ``F[Option[A]]``, optionale Werte in einem Effekt
- **EitherT:** ``F[Either[E, A]]``, Fehlerbehandlung in einem Effekt
- **Kleisli:** ``A => F[B]`` Funktionen komponieren, Dependency Injection
- **Validated:** unabhängige Validierungen, Fehler sammeln (kein flatMap)
- **Ior:** Erfolg mit Warnungen, teilweise Fehler

- **Kategorientheorie** liefert die mathematische Basis für FP-Abstraktionen
- **Higher-Kinded Types** ($F[_]$) erlauben Abstraktion über Container
- **Cats** standardisiert Typklassen: Show, Eq, Order, Functor, Monad, ...
- **Applicative/Validated**: unabhängige Effekte kombinieren, Fehler sammeln
- **Monad Transformers**: verschachtelte Monaden handhabbar mache

Übung

- 8 Teile, aufsteigend in Komplexität
- Jeder Teil baut auf dem vorherigen auf
- Das sbt-Projekt für diese Übung ist vorbereitet (siehe Github)
- Nutze den UserService wenn nötig

- Definiere eine case class `Product(name: String, price: Double)`
- Implementiere `Show[Product]`
- Implementiere `Eq[Product]`:
 - Gleich wenn Name UND Preis identisch
- Teste

```
val p1 = Product("Laptop", 999.99)
val p2 = Product("Laptop", 999.99)
println(p1.show)           // "Produkt: Laptop (€999.99)"
println(p1 == p2)          // true
println(p1 != p2)          // false
```

- Definiere `case class Stats(views: Int, clicks: Int, revenue: Double)`
- Implementiere `Monoid[Stats]`:
 - `empty`: alles auf 0
 - `combine`: Felder addieren
- Teste

```
val day1 = Stats(100, 10, 49.99)
val day2 = Stats(150, 20, 89.99)
println(day1 |+| day2) // Stats(250, 30, 139.98)
```

```
val week = List(day1, day2, Stats(80, 5, 20.0))
println(week.combineAll) // Stats(330, 35, 159.98)
```


- Schreibe eine generische Funktion welche alle Werte verdoppelt.
- Teste mit ``Option(21)``, ``List(1, 2, 3)``, ``Either.right[String, Int](5)``

- Schreibe Validierungsfunktionen
- Hinweis: `condNel`, `ValidatedNel` => `NonEmptyList` => Fehler werden als Liste gesammelt
- Tipp: ``Validated.condNel(bedingung, wert, "Fehlermeldung")``

```
type ValidationResult[A] = ValidatedNel[String, A]
```

```
def validateUsername(name: String): ValidationResult[String] = ???  
// Regeln: nicht leer, mind. 3 Zeichen, nur Buchstaben+Zahlen
```

```
def validateEmail(email: String): ValidationResult[String] = ???  
// Regeln: nicht leer, muss "@" und "." enthalten
```

```
def validateAge(age: Int): ValidationResult[Int] = ???  
// Regeln: zwischen 18 und 120
```

Teil 4b: Validated (Kombination mit mapN)

- Kombiniere die Validierungen zu einem Gesamtergebnis mit mapN
- Teste mit gültigen und ungültigen Daten:

```
def validateUser(name: String, email: String, age: Int): ValidationResult[User] = ???
```

```
validateUser("ab", "invalid", 15)
```

```
// Invalid(NonEmptyList("Mind. 3 Zeichen", "Muss @ enthalten", "Muss >= 18 sein"))
```

```
validateUser("Anna", "anna@mail.de", 25)
```

```
// Valid(User("Anna", "anna@mail.de", 25))
```

Übung

Teil 5: Traverse

- Erstelle eine Liste von Strings und parse sie zu Ints

```
def parseAll(inputs: List[String]): ValidatedNel[String, List[Int]] =  
  inputs.traverse { s =>  
    ???  
  }
```

```
parseAll(List("1", "2", "3"))      // Valid(List(1, 2, 3))  
parseAll(List("1", "abc", "3", "x")) // Invalid(NonEmptyList(...))
```

- Simuliere eine Service-Pipeline:
- Schreibe `findUserEmail(id: Int): Result[String]` mit for-Comprehension
- Teste mit ID 1 (Erfolg) und ID 99 (Fehler)

```
type AppError = String
```

```
type Result[A] = EitherT[Option, AppError, A]
```

- Baue eine Validierungs-Pipeline aus einzelnen Schritten:
- Komponiere: ``val validateName = trim.andThen(nonEmpty).andThen(minLength)``
- Teste mit ``" ", `", ` " ab", ` " Anna "``
- **Aufgabe:** Füge einen Schritt ``toLowerCase`` hinzu

```
type Check[A, B] = Kleisli[Either[String, *], A, B]
```

```
val trim: Check[String, String] = Kleisli(s => Right(s.trim))
```

```
val nonEmpty: Check[String, String] = Kleisli(s =>  
  if (s.nonEmpty) Right(s) else Left("Darf nicht leer sein"))
```

```
val minLength: Check[String, String] = Kleisli(s =>  
  if (s.length >= 3) Right(s) else Left("Mind. 3 Zeichen"))
```

- Baue eine komplette Registrierungs-Funktion mit for-comprehension, nutze den UserService

```
def register(name: String, email: String, age: Int): EitherT[Future, String, String] = {  
  // 1. Kleisli: name und email bereinigen  
  // 2. Validated: alle Regeln prüfen, .toEither.leftMap(_.mkString_("\n"))  
  // 3. Prüfen ob email schon existiert  
  // 4. User einfügen  
  // 5. User.show ausgeben  
}
```